

Universidad Tecnológica Nacional - Facultad Regional Buenos Aires

Sintaxis y Semántica de los Lenguajes

2026

Trabajo Práctico N° 1

Tema: AUTOMATAS

Curso: K2002

Profesora: Roxana Leituz

Fecha Estipulada de Entrega: 27 de Septiembre

Alumno	Legajo
Aldaco, Juan Francisco	222.081-7
Varela, José Manuel	222.767-8

	Fecha	Nota	Observaciones	Fecha Devol.	Firma Docente
Entrega	27/09				
Corrección					

Instructivo

Los tres ejercicios son programas independientes escritos en C, cada uno en su propia carpeta (Ejercicio_1, Ejercicio_2 y Ejercicio_3) dentro de TP_Automatas. Cada carpeta tiene su fuente y, cuando corresponde, el archivo de entrada que usa.

Para compilar y enlazar, parados en la carpeta del ejercicio:

```
gcc -c -o ejercicioN.o ejercicioN.c -Wall  
gcc -o ejercicioN ejercicioN.o -Wall
```

Y para ejecutarlo:

```
./ejercicioN
```

Los ejercicios 1 y 3 arrancan mostrando un menú con dos opciones. La opción 1 pide la cadena por teclado; la opción 2 pide el nombre de un archivo de texto, que tiene que estar en la misma carpeta que el ejecutable (en el repositorio ya viene un entrada.txt de ejemplo en cada uno).

El ejercicio 1 no muestra el resultado por pantalla: lo escribe en el archivo salida.txt, que se crea en la misma carpeta y se puede ver con `cat salida.txt`. Los números de la cadena de entrada van separados por el carácter '@'.

El ejercicio 2 no tiene menú: pide directamente una cadena numérica por teclado y muestra el entero que obtuvo. El ejercicio 3 imprime el resultado de la operación por pantalla, o el mensaje "Vector invalido" si la cadena no pertenece al lenguaje.

Los tres compilan sin advertencias con gcc y la opción -Wall.

Ejercicio 1:

Dada una cadena que contenga varios números que pueden ser decimales, octales o hexadecimales, con o sin signo para el caso de los decimales, separados por el carácter '@', reconocer los tres grupos de constantes enteras, indicando si hubo un error léxico, en caso de ser correcto contar la cantidad de cada grupo.

Para desarrollar este ejercicio antes tenemos que saber qué forma tienen los datos mencionados.

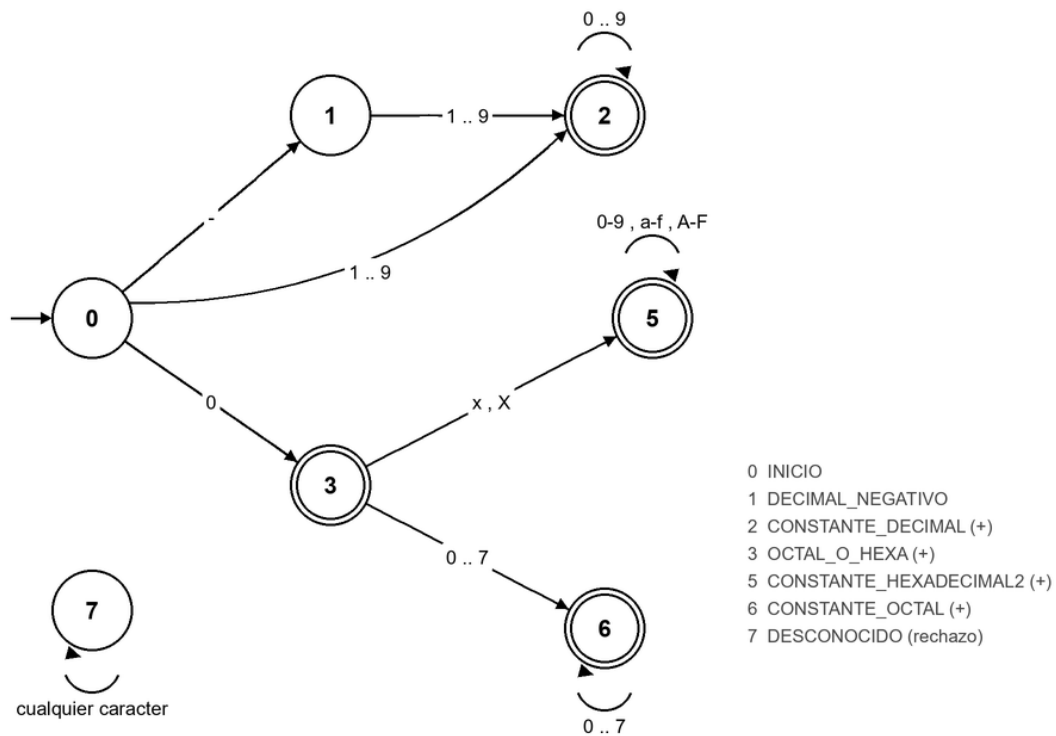
Las constantes hexadecimales comienzan con un cero seguido de una equis (mayúscula o minúscula). Posteriormente puede continuar con un número del cero al nueve o con una letra de la a a la f, tanto en mayúscula como en minúscula. Algunos ejemplos válidos para este tipo de constante son: 0X123, 0x123, 0xAb, 0X0, 0Xa9f.

Las constantes octales tienen la particularidad de que comienzan siempre con 0 y luego tienen números del cero al siete. Algunos ejemplos son: 012, 0034, 076.

Como mención especial consideramos que el 0 pertenece a las constantes octales, aunque también podría incluirse dentro de las decimales. El único motivo de esto es simplificar el código: cuando el usuario ingresa el 0 podemos decir que el número será un octal y quedarnos en ese estado preguntando por los siguientes, salvo que aparezca una equis, en ese caso será un hexadecimal (siempre y cuando el resto de los caracteres también cumplan la condición).

Finalmente, las constantes decimales son todos los números que se pueden formar con los caracteres del cero al nueve, y también incluyen los negativos, es decir los que empiezan con un signo menos. Detrás del signo no puede venir un 0, porque no tendría sentido tener -0 ni -03. Algunos ejemplos de constantes decimales válidas son: 123, -45, 7, -9, 10.

En representación de todo lo anterior se realizó el siguiente autómata con su respectiva tabla.



Los estados marcados con doble círculo son de aceptación. Todo carácter que no figura en una transición lleva al estado 7.

Definido formalmente, este autómata es la quintupla $M1 = (Q, \Sigma, \delta, q_0, F)$, donde:

$Q = \{ 0, 1, 2, 3, 4, 5, 6, 7 \}$

$\Sigma = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f, A, B, C, D, E, F, x, X, - \}$

$q_0 = 0$

$F = \{ 2, 3, 5, 6 \}$

δ es la función de transición que se detalla en la matriz siguiente.

El carácter @ no forma parte del alfabeto: no lo lee el autómata, sino que se usa antes para separar la cadena de entrada en las sub-cadenas que el autómata analiza de a una.

Matriz de transición del autómata del ejercicio 1. El signo - a la derecha del estado indica el estado inicial y el signo + los estados de aceptación.

	-	0	1	2	3	4	5	6	7	8	9	a,A	b,B	c,C	d,D	e,E	f,F	x,X
0-	1	3	2	2	2	2	2	2	2	2	2	7	7	7	7	7	7	7
1	7	7	2	2	2	2	2	2	2	2	2	7	7	7	7	7	7	7
2+	7	2	2	2	2	2	2	2	2	2	2	7	7	7	7	7	7	7
3+	7	6	6	6	6	6	6	6	6	7	7	7	7	7	7	7	7	5
4	7	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	7
5+	7	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	7
6+	7	6	6	6	6	6	6	6	6	7	7	7	7	7	7	7	7	7
7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7

El estado 4 (CONSTANTE_HEXADECIMAL1) está declarado en el enum y tiene sus transiciones definidas en el código, pero ninguna transición lleva hasta él: desde el estado 3 la equis pasa directamente al estado 5. Por ese motivo figura en la matriz pero no en el diagrama.

Algunos detalles sobre el código

Se comenzó realizando la declaración de nombres de constantes con el tipo de dato enum, esto nos permite una mayor expresividad en el código. Estos nombres se usarán más adelante en un switch.

```

13 enum ESTADOS{
14     // Asignar nombres a los estados del autómata
15     INICIO = 0,           //0-
16     DECIMAL_NEGATIVO ,   //1
17     CONSTANTE_DECIMAL ,  //2+
18     OCTAL_O_HEX ,        //3+
19     CONSTANTE_HEXADECIMAL1 , //4
20     CONSTANTE_HEXADECIMAL2 , //5+
21     CONSTANTE_OCTAL ,    //6+
22     DESCONOCIDO
23     //7 (rechazo) // DESCONOCIDO actúa como estado de rechazo
24 };
25 void reconocer(char* valor, FILE* archSalida, int* cant_oct, int* cant_deci, int*
26     cant_hexa);
27 void separar_y_reconocer (char*, FILE* salida);
28 void ingreso_por_archivo (char*, char*);

```

La función main es muy sencilla y se utiliza para llamar al resto de las funciones que harán el trabajo duro del programa; sin embargo, es en este momento que el usuario elige cómo ingresar la cadena deseada. Puede optar por un ingreso en la línea de comandos (realizado con un scanf) o bien mediante un archivo, para esta alternativa además se le pedirá el nombre del archivo y con este se llama a una función.

```
29 int main(){
30
31     int menu;
32     char archivo [20];
33     char vector_palabras [1000];
34     FILE *f_salida = fopen ("salida.txt", "w");
35
36     printf("\n\n..BIENVENIDE AL PROGRAMA DE AUTOMATAS..\n" );
37     printf(" 1) si quiere ingresar la cadena por linea de comandos\n 2) si lo que deasea es
        ingresar la cadena a traves de un archivo txt \n");
38
39     printf("Escriba una opcion: ");
40     scanf ("%d", &menu);
41     system("clear");
42
43     switch (menu){
44         case 1 :
45             printf("Ingrese la cadena de caracteres numericos (Hexadecimales, Ocatales o
                Decimales): ");
46             scanf ("%s", vector_palabras);
47             break;
48         case 2 :
49             printf("Ingrese el nombre del archivo (ej: entrada.txt): ");
50             scanf ("%s", archivo);
51             ingreso_por_archivo(archivo, vector_palabras );
52             break;
53         default:
54             printf ("No ingreso una opcion valida \n" );
55     }
56
57     separar_y_reconocer (vector_palabras, f_salida);
58     // PASO EL ARRAY CON LAS PALABRAS A RECONOCER
59     printf("Archivo creado exitosamente. Para visuiualizarlo escriba: `cat salida.txt`\n" );
60
61     fclose (f_salida);
62     return 0;
63 }

```

```
65 void ingreso_por_archivo (char* archivo, char* vector_palabras) {
66     int i = 0;
67     FILE *f_entrada = fopen(archivo,"r");
68
69     if(f_entrada == NULL){
70         printf("Error en la apertura del archivo.\n" );
71     }
72
73     else {
74         fread(&vector_palabras[i], sizeof(char), 1, f_entrada); //primer elemento
75
76         while(!feof(f_entrada)) {
77             i++;
78             fread(&vector_palabras[i], sizeof(char), 1, f_entrada);
79             //LECTURA DEL ARCHIVO, GUARDO ELEMENTOS DE ENTRADA EN EL VECTOR.
80         }
81
82         fclose (f_entrada);
83     }

```

Cualquiera sea la opción elegida se guarda la cadena en el vector_palabras, que luego es enviado a la función separar_y_reconocer.

Dentro de esta función se crean variables del tipo int que serán pasadas por referencia a una nueva función reconocer, de esta forma reconocer puede ir dejando constancia de la cantidad de cada tipo. Pero antes de ello

se tiene que obtener la sub-cadena de char que representa cada número. Como dice el enunciado, cada una de estas sub-cadenas se encuentra separada por el carácter @; por lo tanto, con la utilización de la función strtok y pasándole como argumento el puntero al vector y el centinela (@) se obtiene cada cadena a analizar.

```

86 //SEPARA CADA ELEMENTO POR '@', EVALUA CON EL AUTOMATA Y LO GUARDA JUNTO A SU TIPO EN EL
    ARCH SALIDA.
87 void separar_y_reconocer (char* vector, FILE* f_salida){
88     int cant_oct = 0, cant_deci = 0, cant_hexa = 0;
89     char* str = NULL;
90     const char* sentinela = "@";
91
92     fprintf(f_salida, "Numero          Cant    Tipo\n" ); //Encabezado
93
94     str = strtok(vector, sentinela);
95     // strtok separa el contenido de un vector en palabras
96     while (str != NULL){
97         // usando el caracter '@' como sentinela. Cada vez que encuentra
98         reconocer(str, f_salida, &cant_oct, &cant_deci, &cant_hexa);
99         // un '@' envía a reconocer la cadena que estaba detrás de ese caracter
100        str = strtok(NULL, sentinela);
101    }
102 }

```

La función reconocer es la más compleja de todas, es la encargada de recorrer cada sub-cadena y decir a qué tipo de constante pertenece. Para esto comienza con un while(1) que se utiliza para que cicle indefinidamente. Dentro de este ciclo tenemos un switch que utiliza como estados los nombres definidos al principio con ayuda de enum. En cada uno de estos estados procura reconocer un carácter y, dependiendo de su contenido, cambia el estado.

Para comenzar tenemos el estado INICIO, en el que se lee el primer carácter de la sub-cadena. Si este carácter es un 0 es posible que el número leído sea un octal o un hexadecimal; es por esto que el estado cambia a OCTAL_O_HEX. Si no lo es, está la posibilidad de que sea un decimal, así que con la función isdigit se pregunta si dicho carácter es un número; si lo es, cambia el estado a CONSTANTE_DECIMAL. Si el carácter leído no es un número puede ser el signo menos, por lo que se envía al estado DECIMAL_NEGATIVO. En cualquier otro caso el estado cambia a DESCONOCIDO: si el estado llega a DESCONOCIDO se imprime que la cadena no fue reconocida y se termina el procesamiento de esa sub-cadena.

```

101 //EVALUA ELEMENTO EN EL AUTOMATA Y LO GUARDA EN EL ARCHIVO SALIDA.
102 void reconocer(char* valor, FILE* f_salida, int* cant_oct, int* cant_deci, int* cant_hexa){
103     int estado = INICIO, i = 0;
104     while(1){
105         switch(estado){
106             case INICIO:
107                 if(valor[i] == '0')
108                     estado = OCTAL_O_HEX;
109                 else if(isdigit(valor[i]))
110                     // isdigit evalua si es un caracter numerico.
111                     estado = CONSTANTE_DECIMAL;
112                 else if (valor[i] == '-'){
113                     estado = DECIMAL_NEGATIVO;
114                 }
115                 else
116                     estado = DESCONOCIDO;
117                 i++;
118                 break;

```

Continuando en orden, tenemos el estado OCTAL_O_HEX. Es en este estado donde, si lo que sigue es un fin de cadena, quiere decir que la cadena terminó y el número era 0 (octal): se guarda en el archivo de salida y retorna rompiendo el ciclo while. Si no, seguimos preguntando si es una equis mayúscula o minúscula; si lo es, todo

indica que el número es un hexadecimal, por lo que el estado cambia a `CONSTANTE_HEXADECIMAL`. En cambio, si lo que se lee es un número entre el 0 y el 7, es un octal (`CONSTANTE_OCTAL`).

```

143         case OCTAL_O_HEXA :
144             if(valor[i] == '\0'){
145                 (*cant_oct)++;
146                 fprintf(f_salida, "%-20s\t %d OCTAL\n", valor, *cant_oct);
147                 return;
148             }
149             else if(valor[i] == 'x' || valor[i] == 'X')
150                 estado = CONSTANTE_HEXADECIMAL2 ;
151             else if(isdigit(valor[i]) && (valor[i] - diferenciaASCII) <= 7)
152                 estado = CONSTANTE_OCTAL ;
153             else
154                 estado = DESCONOCIDO ;
155             i++;
156             break;

```

Otro estado especial es `DECIMAL_NEGATIVO`, al que entramos cuando el primer carácter de la sub-cadena es un signo menos. Ahora preguntamos si el carácter que le sigue es un dígito salvo el 0 (porque no tendría sentido tener -0); si es un dígito, el estado cambia a `CONSTANTE_DECIMAL`.

```

119         case DECIMAL_NEGATIVO : //signo menos
120             if(valor[i] == '\0'){
121                 estado = DESCONOCIDO ;
122             }
123             else if(isdigit(valor[i]) && (valor[i] - diferenciaASCII) > 0)
124                 estado = CONSTANTE_DECIMAL ;
125             else
126                 estado = DESCONOCIDO ;
127             i++;
128             break;

```

Para los siguientes estados es siempre lo mismo. `CONSTANTE_OCTAL`, `CONSTANTE_DECIMAL` y `CONSTANTE_HEXADECIMAL` realizan tres preguntas. Primero, si estoy en ese estado y aparece un `\0` quiere decir que la sub-cadena terminó, por lo que se guarda en el archivo de salida el valor de la constante, el tipo y su cantidad, retornando y saliendo del `while(1)`. Segundo, pregunto si se sigue cumpliendo la condición de que sea octal, decimal o hexadecimal; si la cumple, el estado sigue siendo el mismo y en la próxima vuelta del ciclo pregunto por el carácter siguiente. Tercero y por último, si no cumple con ninguna de las anteriores quiere decir que la cadena NO ES RECONOCIDA y el estado cambia a `DESCONOCIDO`.

```

130         case CONSTANTE_DECIMAL :
131             if(valor[i] == '\0'){
132                 (*cant_deci)++;
133                 fprintf(f_salida, "%-20s\t %d DECIMAL\n", valor, *cant_deci);
134                 return;
135             }
136             else if(isdigit(valor[i]))
137                 estado = CONSTANTE_DECIMAL ;
138             else
139                 estado = DESCONOCIDO ;
140             i++;
141             break;

```

```

158         case CONSTANTE_OCTAL :
159             if(valor[i] == '\0'){
160                 (*cant_oct)++;
161                 fprintf(f_salida, "%-20s\t %d OCTAL\n", valor, *cant_oct);
162                 return;
163             }
164             else if(isdigit(valor[i]) && (valor[i] - diferenciaASCII) <= 7)
165                 estado = CONSTANTE_OCTAL ;
166             else
167                 estado = DESCONOCIDO ;
168             i++;
169             break;

```

Por último podemos mencionar el estado extra de los hexadecimales, CONSTANTE_HEXADECIMAL1, pensado para exigir que después del 0x venga por lo menos un valor más. En el código el estado OCTAL_O_HEXA pasa directamente a CONSTANTE_HEXADECIMAL2 al leer la equis, así que ese estado intermedio quedó sin uso y el autómata implementado es el que muestran el diagrama y la matriz.

```

171         case CONSTANTE_HEXADECIMAL1 :    //Se asegura que luego de la x venga otro valor
172             if(isxdigit(valor[i]))
173                 estado = CONSTANTE_HEXADECIMAL2 ;
174             else
175                 estado = DESCONOCIDO ;
176             i++;
177             break;
178
179         case CONSTANTE_HEXADECIMAL2 :
180             if(valor[i] == '\0'){
181                 (*cant_hexa)++;
182                 fprintf(f_salida, "%-20s\t %d HEXADECIMAL\n", valor, *cant_hexa);
183                 return;
184             }
185             else if(isxdigit(valor[i]))
186                 estado = CONSTANTE_HEXADECIMAL2 ;
187             else
188                 estado = DESCONOCIDO ;
189             i++;
190             break;

```



```

192         case DESCONOCIDO :
193             fprintf(f_salida, "%-20s\tNO RECONOCIDA\n", valor);
194             return;
195     }
196 }

```

Como se puede ver, para la resolución de este problema se tomó la decisión de cambiar el estado en cada uno de los caracteres y de que se retorne inmediatamente el error. Esto permite que, si tengo una cadena que contiene un error al principio, no se continúe procesándola. Por ejemplo, 0Xx es procesada solamente hasta el tercer carácter, ya que entra al estado DESCONOCIDO y allí se hace un return que rompe el while(1).

Ejemplos del programa corriendo

Ingreso por línea de comandos. Primero el menú, que se limpia al elegir la opción:

```
MINGW64:/c/Users/.../TP_Automatas
$ ./ejercicio1

..:BIENVENIDE AL PROGRAMA DE AUTOMATAS:..
1) si quiere ingresar la cadena por línea de comandos
2) si lo que deasea es ingresar la cadena a traves de un archivo txt
Escriba una opcion: 1
```

```
MINGW64:/c/Users/.../TP_Automatas
Ingrese la cadena de caracteres numericos (Hexadecimales, Ocatales o Decimales): 0xFF@127@0159@0xBb1@0Xx@0@010@09@127ABACB@07@-10
Archivo creado exitosamente. Para visualizarlo escriba: 'cat salida.txt'

$ cat salida.txt
Numero          Cant  Tipo
0xFF            1  HEXADECIMAL
127             1  DECIMAL
0159            NO RECONOCIDA
0xBb1           2  HEXADECIMAL
0Xx             NO RECONOCIDA
0              1  OCTAL
010            2  OCTAL
09             NO RECONOCIDA
127ABACB       NO RECONOCIDA
07             3  OCTAL
-10            2  DECIMAL
```

La cadena de entrada viene separada por @ y el resultado queda en salida.txt, con el tipo de cada constante y la cantidad acumulada de cada grupo.

El mismo programa, pero leyendo la cadena desde un archivo .txt:

```
MINGW64:/c/Users/.../TP_Automatas
$ cat entrada.txt
0xFF@127@0159@0xBb1@0Xx@0@010@09@127ABACB@07@-10

$ ./ejercicio1

..:BIENVENIDE AL PROGRAMA DE AUTOMATAS:..
1) si quiere ingresar la cadena por línea de comandos
2) si lo que deasea es ingresar la cadena a traves de un archivo txt
Escriba una opcion: 2
Ingrese el nombre del archivo (ej: entrada.txt): entrada.txt
Archivo creado exitosamente. Para visualizarlo escriba: 'cat salida.txt'
```

Ejercicio 2:

Debe realizar una función que reciba un carácter numérico y retorne un número entero.

Las computadoras solo entienden números, por lo que cuando se escriben caracteres (char) estos tienen una representación numérica basada en ASCII. Esto quiere decir que, por ejemplo, al ingresar el carácter a la computadora lo que recibe es 97, y es en un segundo momento, cuando se realiza la salida del flujo de datos por pantalla, en que se le asigna la letra.

De igual manera sucede con los números. Una cosa es el CARÁCTER 3 y otra el NÚMERO 3. Si guardamos el carácter 3 en una variable char y la imprimimos en pantalla se ve como 3; sin embargo, esa variable char no es más que el número 51 en la tabla ASCII. Al hacer un printf con %c la terminal muestra el carácter, mientras que al imprimirlo con %d podemos ver el valor que tiene dicho carácter internamente (51).

Guiándonos con la siguiente tabla sabemos que, para convertir el carácter en un entero, simplemente debemos restar 48 al valor del char.

'0'	'1'	'2'	'3'	'4'	'5'	'6'	'7'	'8'	'9'
48	49	50	51	52	53	54	55	56	57

Tabla ASCII de los números. Fuente: elcodigoascii.com.ar

Además de tener en cuenta esta propiedad de los char y su representación, para la resolución de este ejercicio tenemos que saber algo elemental de las matemáticas: el valor posicional. En un número, por ejemplo el 234, sabemos que el 4 representa las unidades, el 3 las decenas y el 2 las centenas. Si descomponemos este número obtendremos $2 \times 100 + 3 \times 10 + 4 \times 1$.

Ahora sí, algunos detalles del código. Podemos destacar dos partes de este algoritmo. Una es la resta al carácter numérico char para obtener el valor int; se utilizó un #define llamado diferenciaASCII para dotar de mayor expresividad al código y no dejar el 48 suelto en el medio de la cuenta. La otra mención importante es sobre el valor posicional: al crear el ciclo se comenzó de atrás hacia delante, de esta forma el primer número se multiplica por 1 (unidades), el segundo por 10 (decenas) y el tercero por 100 (centenas). En cada vuelta la variable VP se multiplica por 10 y el número calculado finalmente se retorna al main.

```
26 #define diferenciaASCII 48
27
28 #include <stdio.h>
29 #include <string.h>
30
31 int my_atoi(char *);
32
33 int main()
34 {
35     int numero;
36     char string [100];
37
38     printf("Ingrese un string numerico: ");
39     scanf("%s", string);
40
41     numero = my_atoi(string);
42
43     printf("El string ingresado es: '%s' \n", string);
44     printf("La conversion se realizo exitosamente, el NUMERO (int) ingresado es: %d\n", numero);
45
46     return 0;
47 }
```

```
49 int my_atoi(char* string){
50
51     int largo, i, numero = 0;
52     int VP = 1;    // Valos posicional 1, 10, 100, 1000 ...
53
54     largo = strlen(string);
55
56     for (i = (largo-1) ; i >= 0 ; i--) {
57         // uso largo-1 porque el indice del vector comienza en 0
58         numero += (string[i] - diferenciaASCII) * VP;
59         VP *= 10; // Se utiliza para calcular unidades, decenas, centenas ...
60     }
61
62     return numero;
63 }
```

El main es solamente un programa de prueba: pide una cadena numérica por teclado, la pasa por la función y muestra la cadena ingresada junto al número obtenido.

```
MINGW64:/c:/Users/.../TP_Automatas
$ ./ejercicio2
Ingrese un string numerico: 3
El string ingresado es: '3'
La conversion se realizo exitosamente, el NUMERO (int) ingresado es: 3

$ ./ejercicio2
Ingrese un string numerico: 123
El string ingresado es: '123'
La conversion se realizo exitosamente, el NUMERO (int) ingresado es: 123

$ ./ejercicio2
Ingrese un string numerico: 2045
El string ingresado es: '2045'
La conversion se realizo exitosamente, el NUMERO (int) ingresado es: 2045
```

Cada carácter se convierte con la resta de 48 y se acomoda según su valor posicional.

Razonamiento utilizado

Antes de comenzar con la resolución del ejercicio se explicará el razonamiento utilizado. Se parte de un vector de caracteres que denominaremos cadena. Tomemos como ejemplo la operación $12+3+4-5+6*7$.

1	2	+	3	+	4	-	5	+	6	*	7
---	---	---	---	---	---	---	---	---	---	---	---

cadena

Una función que se desarrolló se encarga de diferenciar los números de los operadores y separarlos en distintos vectores. Para ello utilizamos `v_num`, que es un vector de `int`, y `v_ope`, que es un vector de `char`.

Índice	0	1	2	3	4	5	6	...	999
<code>v_num</code>	12	3	4	5	6	7			
<code>v_ope</code>	+	+	-	+	*	\0			

Ahora bien, si vemos en detalle podemos apreciar cierta relación entre los índices. El `*` está en el índice (4) de `v_ope`, y lo que debería hacer es operar el 6 con el 7, que tienen los índices (4) y (5) respectivamente en `v_num`. Lo mismo pasa con el `-`: su índice en `v_ope` es (2) y el del 4 y el 5 en `v_num` es (2) y (3). Es decir, cada operador se relaciona con dos números, y los índices de esos números son: el mismo que el del operador y el siguiente.

Ahora bien, cuando se realiza una operación podemos decir que los números se consumen. Dentro de nuestra función vamos realizando semi-resultados de la cadena ingresada. Cada uno de estos semi-resultados, o resultados parciales, se guarda dentro del vector de números. ¿En qué posición? En la del índice de la operación, es decir, en el índice del primer dígito que estoy operando; y para no repetir números ponemos un 0 en la posición del otro de los números.

Siguiendo con el ejemplo, primero se resuelven los productos. El `*` está en el índice (4), así que se multiplica el número de la posición (4) con el de la siguiente (5): $6*7 = 42$. El resultado se guarda donde antes estaba el 6 (primer término) y en el lugar del 7 (segundo término) se coloca un 0 para no volver a operar con dicho valor.

Índice	0	1	2	3	4	5	6	...	999
<code>v_num</code>	12	3	4	5	$6*7 = 42$	0			

Después se recorren las sumas y las restas de izquierda a derecha, haciendo lo mismo:

Índice	0	1	2	3	4	5	6	...	999
<code>v_num</code>	$12+3 = 15$	0	4	5	42	0			
<code>v_num</code>	15	$0+4 = 4$	0	5	42	0			
<code>v_num</code>	15	4	$0-5 = -5$	0	42	0			
<code>v_num</code>	15	4	-5	$0+42 = 42$	0	0			
<code>v_num</code>	15	4	-5	42	0	0			

Una vez que se realizan todas las operaciones del vector `v_ope` nos queda `v_num` con todos los resultados parciales. Lo único que queda por realizar es la suma de todos ellos: $15 + 4 - 5 + 42 = 56$, que es el resultado de la operación escrita en la cadena.

Algunos detalles sobre el código

Al igual que en el punto 1 se tiene un main bastante breve, donde se da a elegir si se quiere ingresar la cadena por línea de comandos o por un archivo.

```
15 enum ESTADOS {
16     INICIO = 0,           //Q1
17     NUMERO,               //Q3
18     NUMERO_O_OPERADOR ,  //Q2
19     RECHAZO
20 };
21
22 //PROTOTIPOS DE FUNCIONES
23 int isoperator(char);
24 int validar_vector (char *);
25 int my_atoi(char* string);
26 void ingreso_por_archivo (char* archivo, char* cadena);
27 void separar_numeros_de_operadores (char* cadena, char* v_ope, int* v_num, int* largo);
28 int calcular (char* v_ope, int* v_num, int largo);
```

```
31 int main(){
32
33     int menu, resultado;
34     char name_file [20];
35     char cadena [1000], v_ope[1000];
36     int v_num [1000];
37     int largo;
38
39     printf("\n\n..BIENVENIDE AL PROGRAMA DE AUTOMATAS..\n" );
40     printf(" 1) si quiere ingresar la cadena por linea de comandos\n 2) si lo que deasea es
        ingresar la cadena a traves de un archivo txt \n");
41
42     printf("Escriba una opcion: " );
43     scanf ("%d", &menu);
44     system("clear");
45
46     switch (menu){
47         case 1 :
48             printf("Ingrese la cadena de caracteres numericos (Hexadecimales, Ocatales o
                Decimales): ");
49             scanf ("%s", cadena);
50             break;
51
52         case 2 :
53             printf("Ingrese el nombre del archivo (ej: entrada.txt): " );
54             scanf ("%s", name_file);
55             ingreso_por_archivo (name_file, cadena);
56             break;
57
58         default:
59             printf ("No ingreso una opcion valida \n" );
60             return 1;
61     }
62
63     if (validar_vector (cadena)){
64
65         separar_numeros_de_operadores (cadena, v_ope, v_num, &largo);
66         // largo es la cantidad de elementos ocupados en v_num
67         resultado = calcular(v_ope, v_num, largo);
68         printf("El Resultado es: %d \n", resultado);
69     }
70
71     else
72         printf("Vector invalido\n" );
73
74     return 0;
75 }
```

Posteriormente se valida la cadena ingresada; para ello se utilizó el autómata mostrado al inicio. La función `isoperator` también fue desarrollada, únicamente recibe un `char` y devuelve 1 para los casos afirmativos y 0 en caso de no ser ninguno de los operadores.

```

136 //AUTOMATA
137 int validar_vector(char* cadena) {
138     int estado = INICIO, i=0;
139     while(1){
140         switch(estado){
141             case INICIO: //Q1
142                 if(isdigit(cadena[i]))
143                     estado = NUMERO_O_OPERADOR ;
144                 else
145                     estado = RECHAZO;
146                 i++;
147                 break;
148
149             case NUMERO_O_OPERADOR : //Q2
150                 if (cadena[i] == '\0')
151                     return True; //valido
152                 else if(isdigit(cadena[i]))
153                     estado = NUMERO_O_OPERADOR ;
154                 else if(isoperator(cadena[i]))
155                     estado = NUMERO; //Despues de un operador viene si o si un numero
156                 else
157                     estado = RECHAZO;
158                 i++;
159                 break;
160
161             case NUMERO: //Q3
162                 if(isdigit(cadena[i]))
163                     estado = NUMERO_O_OPERADOR ;
164                 else
165                     estado = RECHAZO;
166                 i++;
167                 break;
168
169             case RECHAZO:
170                 return False; //invalido
171         }
172     }
173 }

175 int isoperator (char c){
176     if (c == '+' || c == '-' || c == '*' )
177         return True;
178     return False;
179 }

```

Una vez que la cadena está validada se separan los números de los operadores en v_num y v_ope. El prototipo de la función que realiza esto es:

```
void separar_numeros_de_operadores(char* cadena, char* v_ope, int* v_num, int* largo);
```

No hay mucho que explicar sobre esta función más que mencionar que se le pasa por referencia la variable largo, para saber la cantidad de elementos que tiene el vector de números. Los números se obtienen con strtok usando los operadores como centinelas y se convierten con my_atoi.

```
78 void separar_numeros_de_operadores (char* cadena, char* v_ope, int* v_num, int* largo){
79
80     char copy_cadena [1000]; // Cuando uso strtok pierdo el string
81     char *token;
82     int i=0, j=0;
83
84     memset(v_num, 0, 1000); // LIMPIO EL VECTOR
85     memset(v_ope, 0, 1000); // LIMPIO EL VECTOR
86
87     strcpy(copy_cadena, cadena);
88
89     //GUARDO LOS NUMEROS EN v_num
90     token = strtok(copy_cadena, "+-");
91     while (token != NULL){
92
93         v_num[i] = my_atoi(token);
94         //Convierto el string a int y lo guardo en v_num (funcion del punto 2)
95         token = strtok(NULL, "+-");
96         i++;
97     }
98     (*largo) = i; //Necesito saber cuantas posiciones del vector ocupé
99
100    //GUARDO LOS OPERADORES EN v_ope
101    for(i=0; i < (strlen(cadena)); i++){
102        if(isoperator(cadena[i])){
103            v_ope[j] = cadena[i];
104            j++;
105        }
106    }
107 }
```

```
201 //Convierte una cadena de numeros a enteros (punto 2)
202 int my_atoi(char* string){
203
204     int largo, i, numero = 0;
205     int CDU = 1;    // Clasificación Decimal Universal 1, 10, 100, 1000 ...
206
207     largo = strlen(string);
208
209     for (i = (largo-1) ; i >= 0 ; i--) {
210         // uso largo-1 porque el indice del vector comienza en 0
211         numero += (string[i] - diferenciaASCII) * CDU;
212         CDU *= 10; // Se utiliza para calcular unidades, decenas, centenas ...
213     }
214     return numero;
215 }
```

Una vez separados los números de los operadores se pasan ambos vectores a la función calcular. Esta devuelve el resultado de la cadena ingresada anteriormente.

```

108 int calcular (char* v_ope, int* v_num, int largo) {
109     int i, resultado = 0;
110
111     for(i=(strlen(v_ope) - 1); i >= 0; i--){
112         // recorro de atras para adelante por si tengo multiples productos no multiplicar
113         pot 0
114         if(v_ope[i] == '*'){
115             //Primero resuelvo el * para respetar precedencia
116             v_num[i] = v_num[i] * v_num[i+1];
117             v_num[i+1] = 0;
118         }
119     }
120
121     for(i=0; i < strlen(v_ope); i++){
122         if(v_ope[i] == '+'){
123             v_num[i] = v_num[i] + v_num[i+1];
124             v_num[i+1] = 0;
125         }
126         if(v_ope[i] == '-'){
127             v_num[i] = v_num[i] - v_num[i+1];
128             v_num[i+1] = 0;
129         }
130     }
131
132     for(i=0; i < largo; i++){
133         resultado += v_num[i];
134     }
135
136     return resultado;
137 }

```

Si vemos más de cerca el primer for, es el que resuelve los productos antes que las sumas y las restas, y así se respeta la precedencia de operadores. El otro detalle importante es que se recorre de atrás hacia adelante. ¿Por qué? Porque, como vimos anteriormente, cuando se resuelve una operación guardamos el valor del resultado en el índice del primer número y colocamos un cero en el índice siguiente. Si tenemos una multiplicación sucesiva y se recorre de izquierda a derecha, el siguiente producto se realizaría con valor 0. Por ejemplo, $2*3*4 = 24$.

Si recorremos de izquierda a derecha:

Índice	0	1	2
v_num	2	3	4
v_num	$2*3 = 6$	0	4
v_num	6	$0*4 = 0$	0

Mientras que si lo recorremos de derecha a izquierda:

Índice	0	1	2
v_num	2	3	4
v_num	2	$3*4 = 12$	0
v_num	$2*12 = 24$	0	0

Al poner 0 en la posición siguiente hacemos que el producto que sigue realice $0*4 = 0$, por eso el recorrido es de atrás hacia adelante.

A continuación se analizan las sumas y las restas, esta vez de izquierda a derecha. Si es un más se hace la suma del número y el siguiente, y se pone un 0 en la última posición; si es un menos se realiza la resta entre el número y el siguiente. Finalmente, un for recorre el vector de enteros sumando los resultados parciales y se retorna dicho valor.

Ejemplos del programa corriendo

Programa corriendo con ingreso por línea de comando:

```
MINGW64:/c/Users/.../TP_Automatas

Ingrese la cadena de caracteres numericos (Hexadecimales, Ocatales o Decimales): 3+4*7+3-5
El Resultado es: 29

Ingrese la cadena de caracteres numericos (Hexadecimales, Ocatales o Decimales): 2*3*4
El Resultado es: 24

Ingrese la cadena de caracteres numericos (Hexadecimales, Ocatales o Decimales): 12+3+4-5+6*7
El Resultado es: 56
```

El mismo programa, pero corrido con un archivo .txt como entrada:

```
MINGW64:/c/Users/.../TP_Automatas

$ cat entrada.txt
3+4*7+3-5

$ ./ejercicio3

...BIENVENIDE AL PROGRAMA DE AUTOMATAS...
1) si quiere ingresar la cadena por linea de comandos
2) si lo que deasea es ingresar la cadena a traves de un archivo txt
Escriba una opcion: 2
Ingrese el nombre del archivo (ej: entrada.txt): entrada.txt
El Resultado es: 29
```

Prueba de validación del vector con el autómata:

```
MINGW64:/c/Users/.../TP_Automatas

1er caso: operador al comienzo de la cadena.
Ingrese la cadena de caracteres numericos (Hexadecimales, Ocatales o Decimales): +3+4
Vector invalido

2do caso: operador al final de la cadena.
Ingrese la cadena de caracteres numericos (Hexadecimales, Ocatales o Decimales): 3+4+
Vector invalido

3er caso: operador repetido.
Ingrese la cadena de caracteres numericos (Hexadecimales, Ocatales o Decimales): 3++4
Vector invalido
```